Regularization Techniques in Deep Learning

Computer Vision

Week 5 – Lecture 9

Dropout | Weight Decay | Batch Normalization | Data Augmentation

Recap & Motivation

Quick Recap:

- CNN architectures became progressively deeper (LeNet → AlexNet → VGG → ResNet → Inception).
- Deeper models have significantly greater learning capacity and representational power.
- But greater capacity also increases the massive risk of **overfitting**.

The Critical Question:

"If a model has enough parameters to perfectly memorize the training data, will it perform well on brand new, unseen images?"

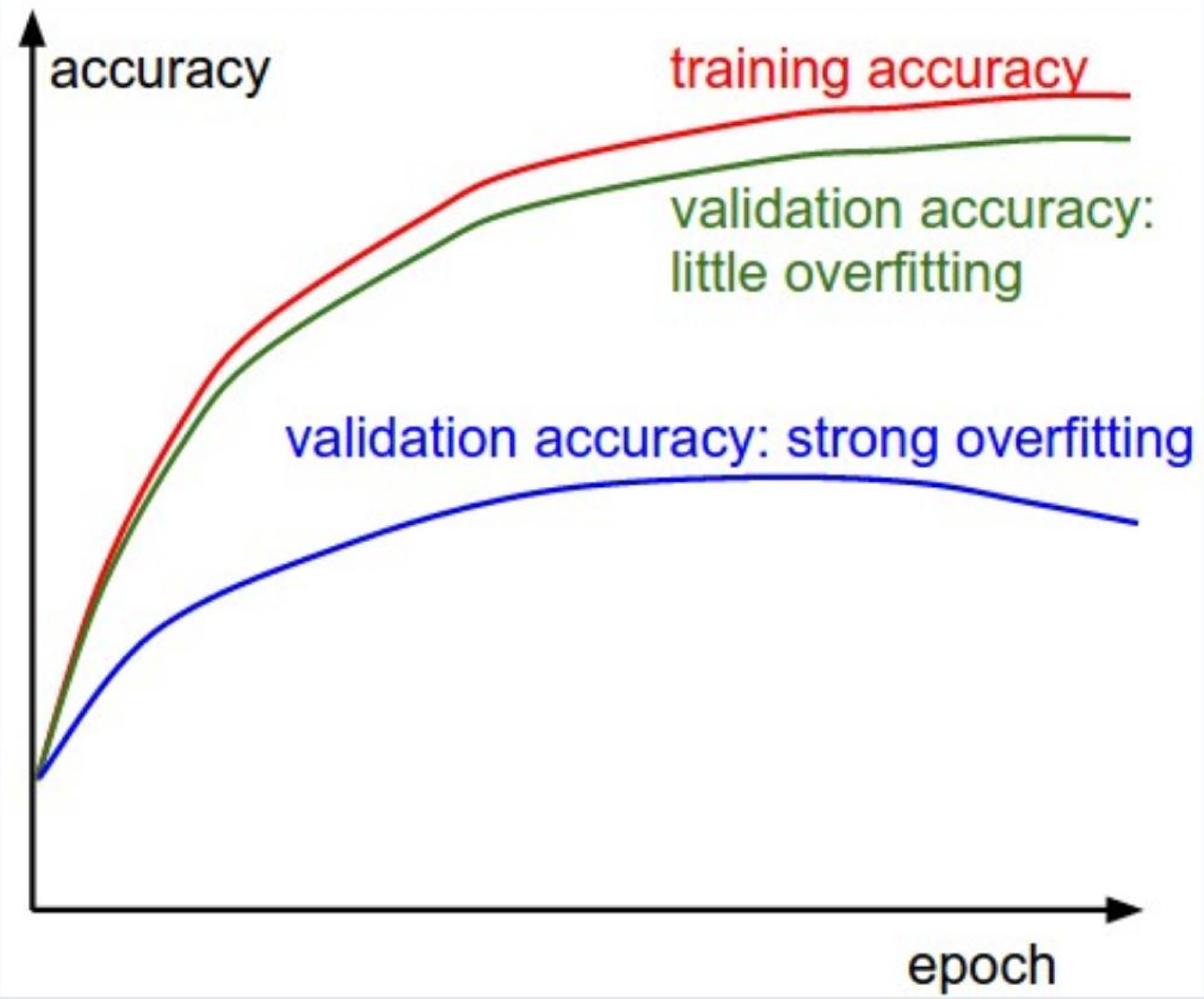
What is Overfitting?

Overfitting occurs when a neural network learns the *noise* in the training data rather than the underlying patterns.

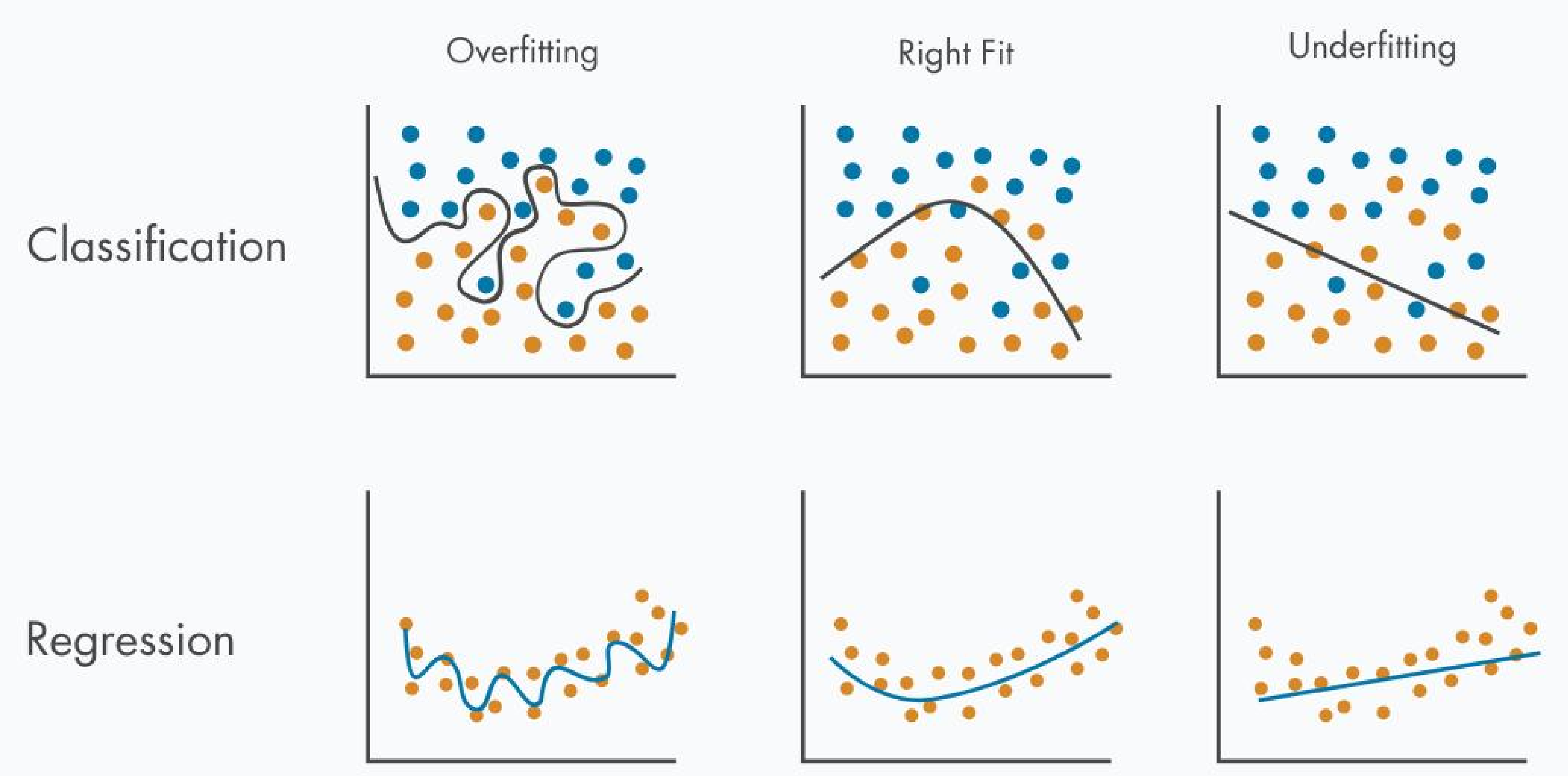
- **Training Data:** The model achieves excellent, near-perfect accuracy on the data it was trained on.

  Test Data: The model performs poorly on unseen validation or test data.

- **Visualizing the Problem:** Watch the graph. While training accuracy (or loss) continues to improve, the validation accuracy plateaus and then worsens. The gap between them is the magnitude of overfitting.



Model Fit: Under to Over



- **Underfitting:** The model is too simple. It cannot capture the underlying trend. Both Training and Test accuracy are low.
- ✓ **Proper Fitting (Robust):** The model captures the core pattern smoothly, ignoring random noise. High Training and high Test accuracy.
- ✗ **Overfitting:** The model is too complex. It bends to capture every single anomaly. Extremely high Training accuracy, but low Test accuracy.

> **"The goal is not maximum training accuracy—it is maximum generalization."**

What is Regularization?

Regularization consists of techniques that deliberately constrain or modify the learning process to reduce overfitting and improve generalization.

Model Memorization (Overfitting) → Apply Regularization (Constraints) → Better Generalization (Real-world Success)

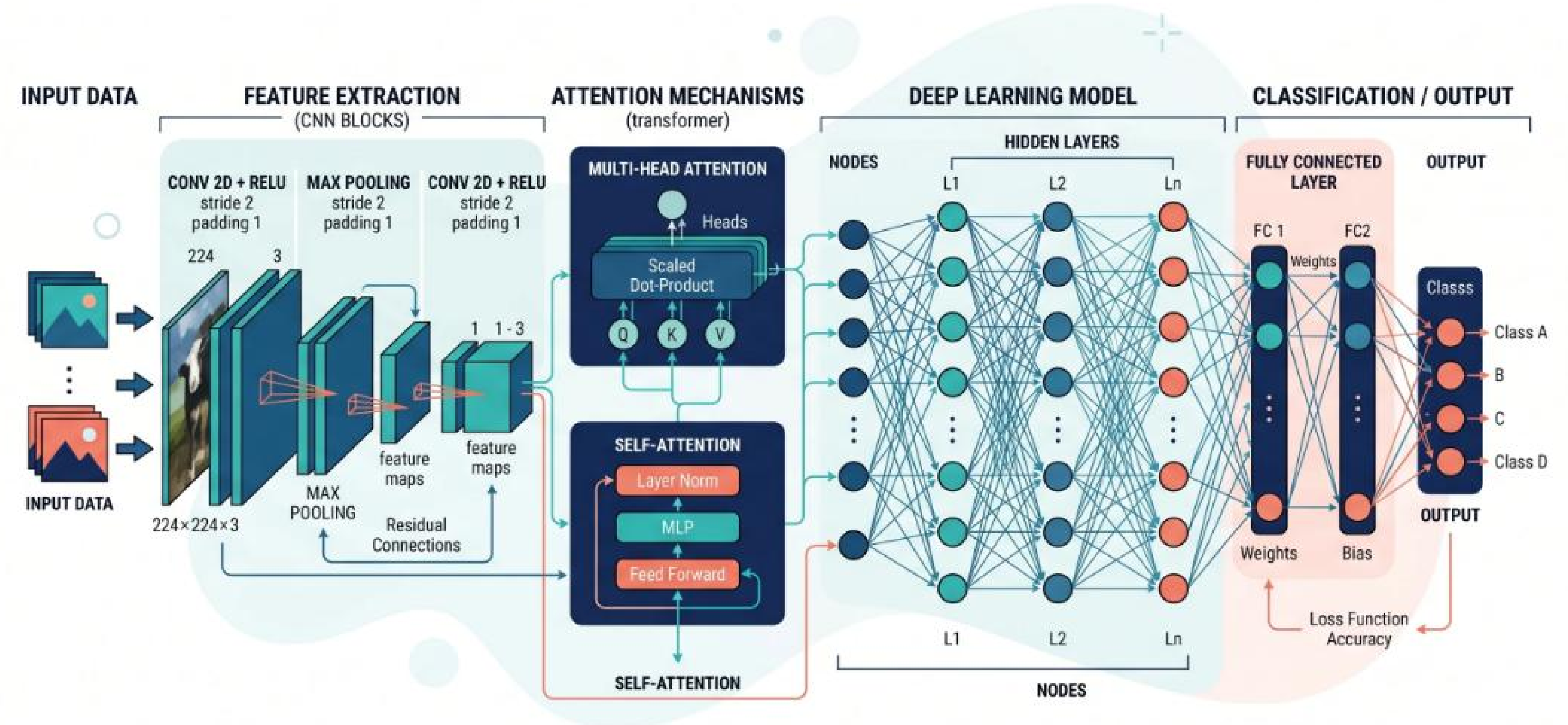
Today's Four Core Techniques:

- Dropout
- Weight Decay
- Batch Norm
- Data Augmentation

Dropout: The Core Idea

Proposed by Geoffrey Hinton et al. (2012), Dropout is a surprisingly simple yet highly effective technique.

- **The Concept:** Randomly deactivate (or "drop out") a subset of neurons during each training step.
- **The Effect:** It physically cuts the connections for that specific forward and backward pass, preventing neurons from co-adapting.
- **Active only during training:** During inference/testing, all neurons are turned back on (scaled appropriately).



How Dropout Works

Standard Network

All neurons are active and fully connected in every single iteration.

With Dropout (e.g., p=0.5)

Randomly selected neurons are dropped out in each forward pass.

Common Dropout Rates:

0.2 to **0.3** (for Convolutional Layers) | **0.5** (for Dense Fully-Connected Layers)

Each iteration trains a slightly different architecture!

Why Dropout Works

- **Prevents Co-adaptation:** Neurons cannot rely on other specific neurons to fix their mistakes.
- **Robust Features:** Forces each neuron to learn useful, independent features on its own.
- **Ensemble Effect:** Approximates training a large number of neural networks and averaging their results.

The Cricket Analogy

"A cricket team shouldn't depend on only one star player. If Rohit gets out, the rest of the team must still know how to score runs."

Dropout forces every neuron to be capable, rather than relying on a few 'star' neurons.

Weight Decay (L2 Regularization)

Instead of removing neurons, what if we just penalized them for being too dominant?

- **The Concept:** Add a penalty term to the loss function based on the size of the weights.
- **The Goal:** Discourage the network from learning extremely large weight values.
- **Lambda (λ):** The hyperparameter controlling how severely we punish large weights.

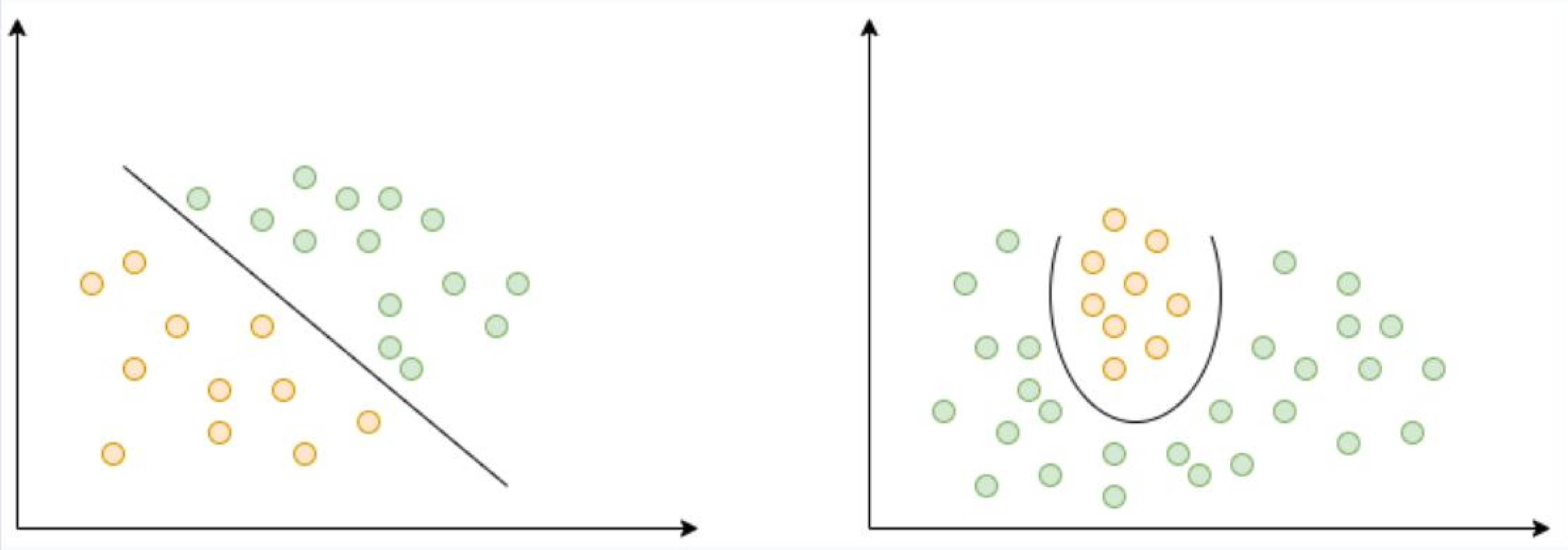
The Modified Loss Function

$$\text{Loss} = L_{\text{original}} + \lambda \times \sum_i w_i^2$$

L_{original}
Error on Data

$\lambda \Sigma w^2$
Complexity Penalty

Intuition Behind Weight Decay



Large Weights (No Regularization): Allows the model to create highly complex, sharp, and erratic decision boundaries to perfectly fit every data point.

Small Weights (With Weight Decay): Forces the network to use smoother, more gradual functions.

Simplicity over Complexity: It adheres to Occam's Razor—simpler models are preferred. We shrink the weights without entirely removing neurons (unlike Dropout).

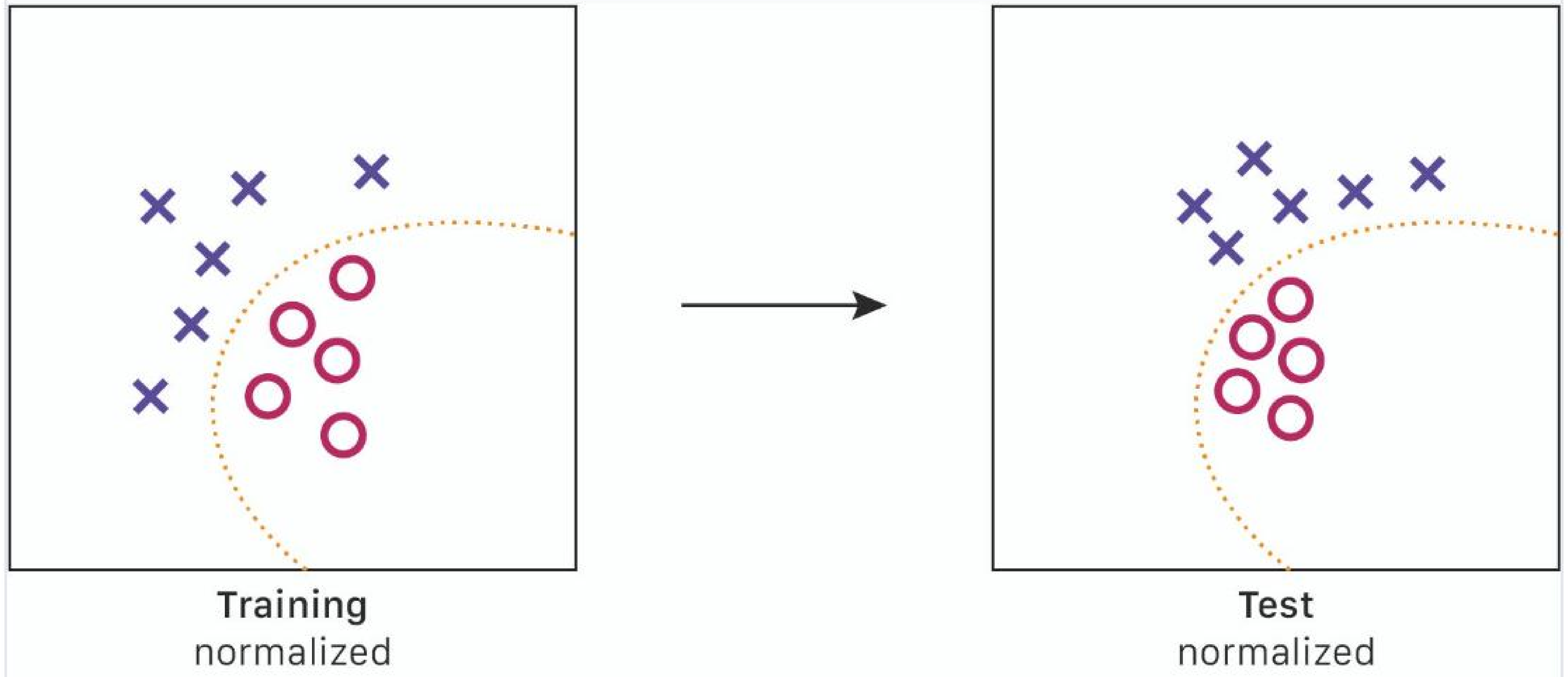
Batch Norm: Why Do We Need It?

The Problem: Internal Covariate Shift

- **Shifting Targets:** As weights in early layers change during training, the distribution of inputs to deeper layers constantly shifts.

  Slow Training: Deeper layers must continuously adapt to this shifting data, causing training to be slow and unstable.

- **Sensitivity:** Models become extremely sensitive to how their weights were initially randomized.



How Batch Normalization Works

Inserted directly inside the network, usually between the Convolution and the ReLU Activation.

1. Input Mini-Batch
(Raw feature maps)

→

2. Normalize
(Force Mean=0, Variance=1)

→

3. Scale & Shift
(Learn parameters γ and β)

→

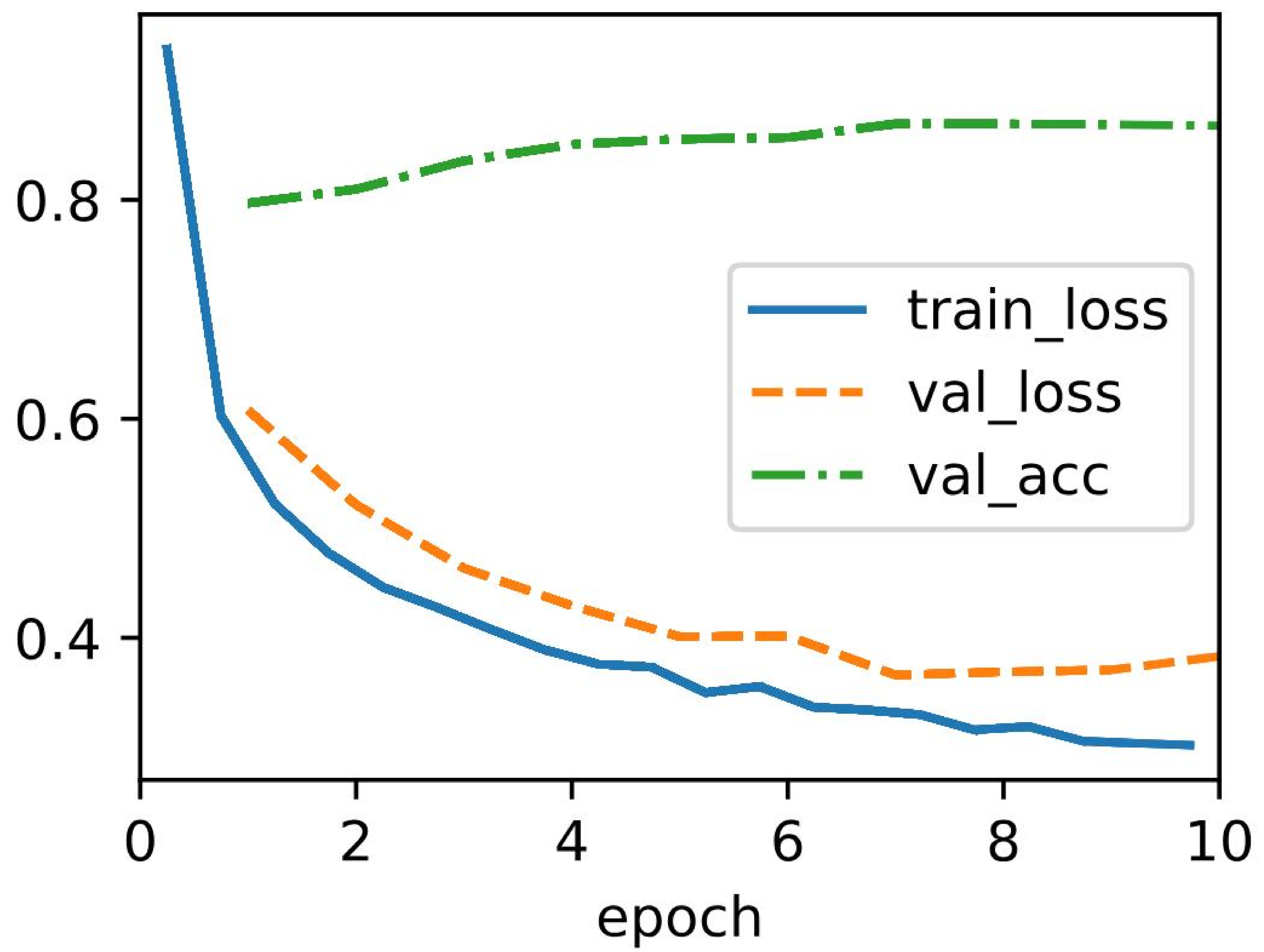
4. Activation
(Pass to ReLU)

Why Scale and Shift?

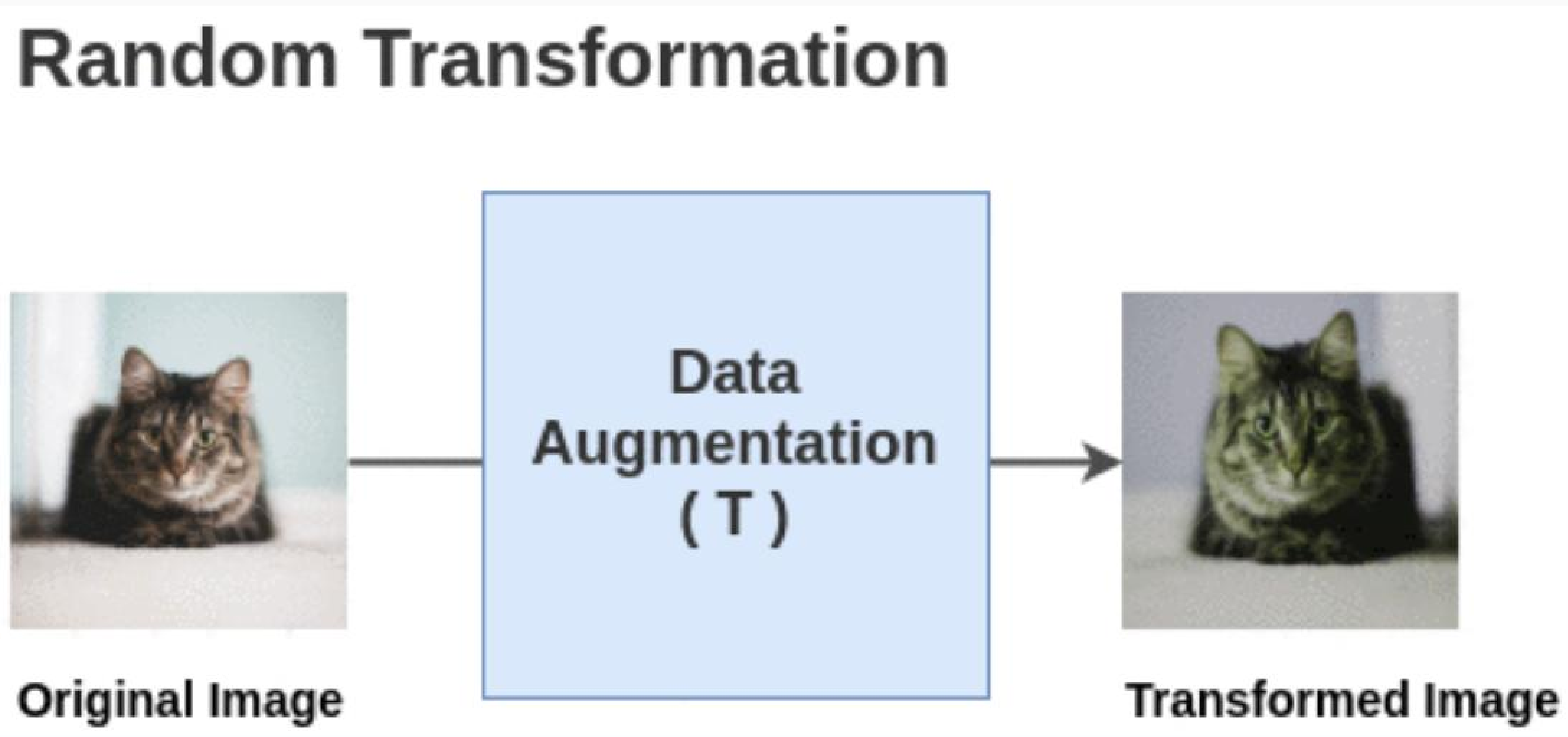
If forcing the data to mean=0 and variance=1 ruins the feature representation, the network learns the gamma (γ) and beta (β) parameters to scale and shift the data back to an optimal distribution.

Advantages of Batch Normalization

- **Faster Convergence:** Reaches high accuracy in far fewer epochs.
- **Higher Learning Rates:** You can train aggressively without the network gradients exploding or vanishing.
- **Mild Regularization Effect:** Because batches are randomly mixed, the mean/variance acts as slight noise, reducing the need for heavy Dropout.
- **Less Initial Sensitivity:** Fixes issues caused by poor initial weight randomization.



Data Augmentation Strategy



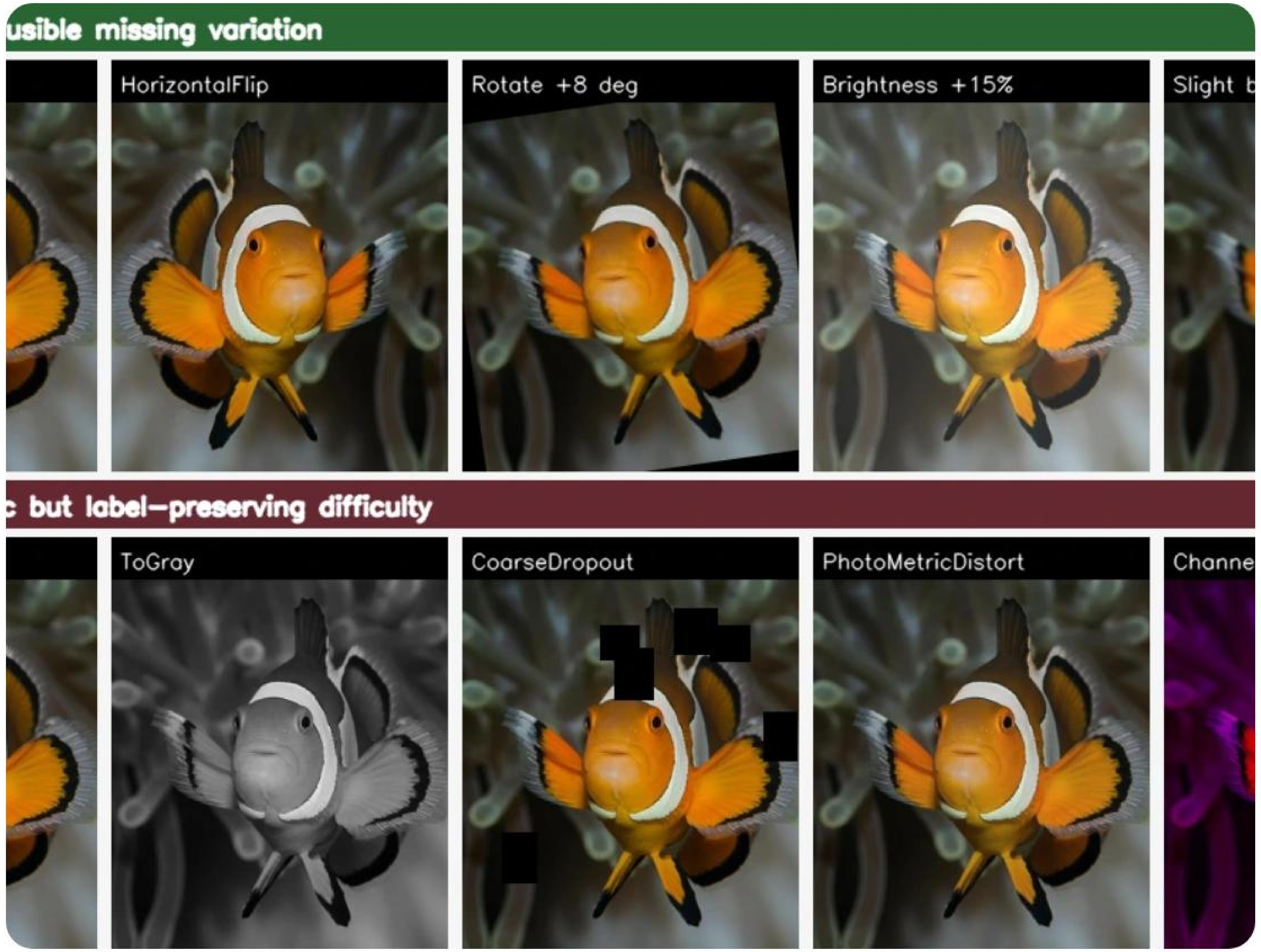
The Best Regularizer is More Data!

- **The Concept:** Instead of spending millions to collect more real images, we artificially create new variations from our existing dataset.
- **Dynamic Generation:** Transformations are usually applied 'on-the-fly' in CPU memory right before being fed to the GPU.
- **The Benefit:** Massively expands dataset size and makes the network robust to spatial variations.

Common Augmentation Techniques

- Horizontal Flip
- Rotation (± degrees)
- Random Cropping
- Zoom In/Out
- Translation (Shift)
- Brightness Adjust
- Contrast Adjust
- Noise Injection

By combining these randomly, one source image can yield thousands of unique training samples.



Why Data Augmentation Works

Forces Invariance

It teaches the model that "a rotated cat is still a cat." The network learns features invariant to position, scale, and lighting.

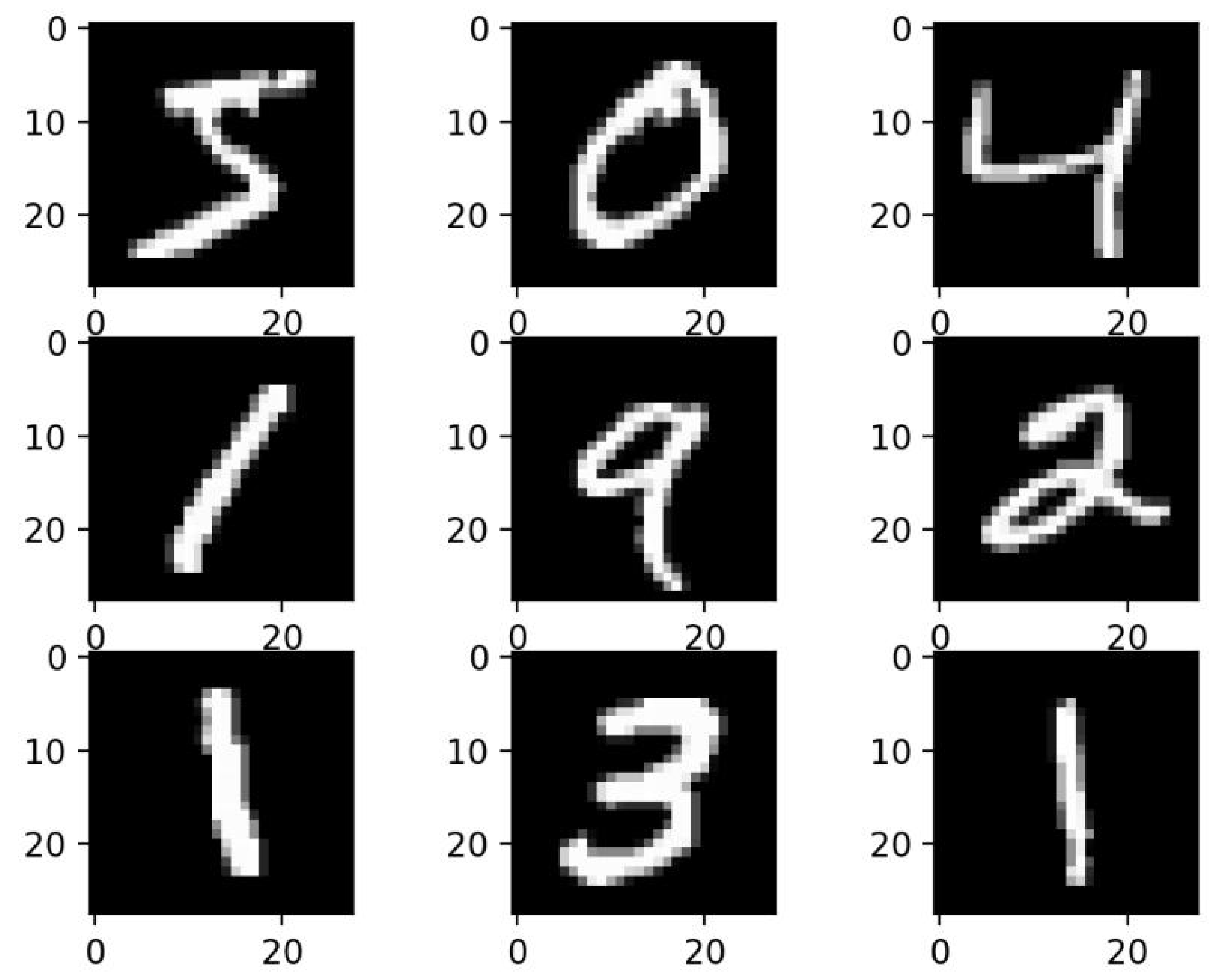
Stops Pixel Memorization

If the background shifts every epoch, the network is forced to look at the actual object rather than background context.

Fills Data Gaps

Simulates real-world imperfect conditions (blur, low light) that might be missing from clean dataset photos.

When Augmentation Can Be Harmful



Domain Knowledge Matters!

- **Digit Recognition (OCR):** Rotating a '6' by 180 degrees turns it into a '9'. Horizontal flips destroy text meaning.
- **Medical Scans:** Applying random distortions or heavy color shifting to tumor scans can destroy the actual pathology you want to detect.
- **Excessive Cropping:** You might completely crop the object out of the image, feeding the network a label for a background block of sky.

Comparing Regularization Methods

Technique	Main Idea	Where Applied	Reduces Overfitting?	Training Speed
Dropout	Randomly disable neurons	Dense / Final Conv Layers	✓ Yes (Strong)	Slightly Slower
Weight Decay	Penalize large weights	Loss Function	✓ Yes	No Impact
Batch Norm	Normalize layer inputs	Between Conv and ReLU	✓ Yes (Mild)	**Much Faster**
Data Augmentation	Create artificial data variations	Input Pipeline	✓ Yes (Very Strong)	Slower (CPU bound)

Regularization in Modern CNNs

Modern architectures combine these techniques into a single, cohesive training pipeline.

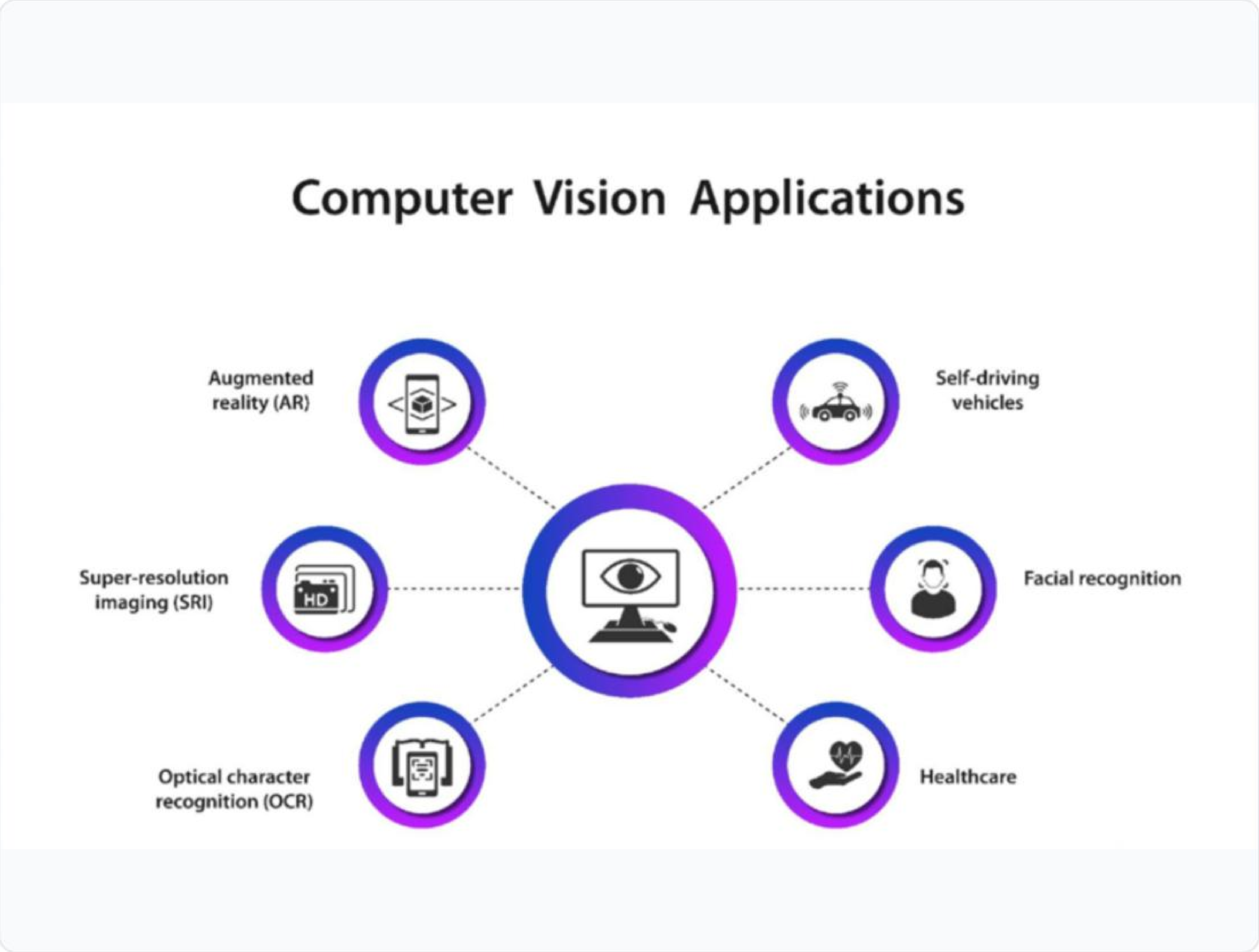
1. Raw Image Loader → 2. Data Augmentation → 3. Convolution & Batch Norm → 4. Dropout (Dense) → 5. Weight Decay in Loss

"We don't just pick one technique—we use them together to build a comprehensive defense against overfitting."

Real-World Applications

Where overfitting is absolutely unacceptable:

- **Medical Scans:** Memorizing hospital watermarks instead of learning tumor shapes leads to misdiagnoses.
- **Autonomous Driving:** A car that overfits to sunny days will crash in rain or snow. Augmentation (weather simulation) is vital.
- **Face Recognition:** Models must handle sunglasses, varying lighting, and angles—enforced by robust regularization.
- **Defect Detection:** Industrial models must detect novel scratches, not just the exact scratch patterns they were trained on.



Key Takeaways

Dropout

Removes neurons temporarily during training to prevent co-adaptation and enforce independent feature learning.

Weight Decay

Modifies the loss function to penalize large weights, encouraging simpler and smoother decision boundaries.

Batch Normalization

Normalizes data between layers to stabilize shifting distributions, allowing much faster training convergence.

Data Augmentation

Dynamically alters training images (rotations, crops) to artificially expand the dataset and teach visual invariance.

Discussion Questions

1. Why doesn't higher training accuracy always mean a better model?
2. How is the mechanism of Dropout different from Weight Decay?
3. Why is Batch Normalization not just a simple normalization technique (hint: Gamma & Beta)?
4. Which data augmentation would be inappropriate for OCR text detection?
5. If you only had 500 training images, which regularization technique would help the most?

Next Lecture (Week 5 – Lecture 10)

Training Mechanics

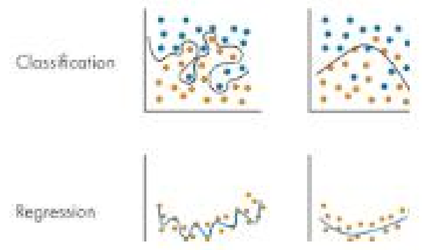
Weight Initialization
Learning Rate Scheduling
Early Stopping

(Followed by Practical 4 Capstone)

Image Sources

https://cs231n.github.io/assets/nn3/accuracies.jpeg

Source: cs231n.github.io

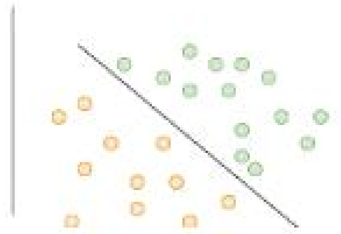


https://www.mathworks.com/discovery/overfitting/_jcr_content/mainParsys/image.adapt.full.medium.svg/1781759708735.svg

Source: www.mathworks.com

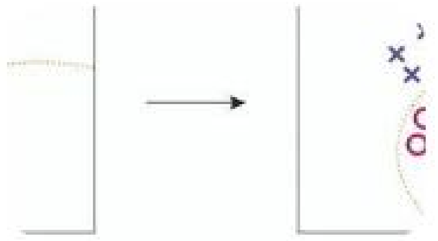
https://conceptviz.app/images/blog/how-to-draw-neural-network-architecture-diagram-guide.webp

Source: conceptviz.app



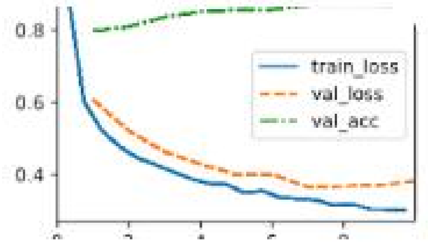
https://miro.medium.com/1*fGFBweRC2fvJypkq_OJelg.png

Source: medium.com



https://towardsdatascience.com/wp-content/uploads/2020/11/1jGTvhWXnltmk_9MWMdRPGQ.jpeg

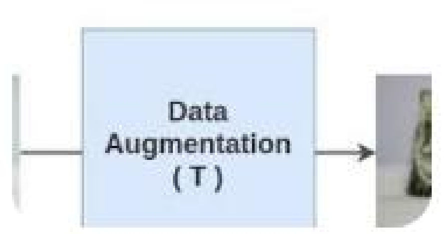
Source: towardsdatascience.com



https://d2l.ai/_images/output_batch-norm_cf033c_110_0.svg

Source: d2l.ai

Image Sources

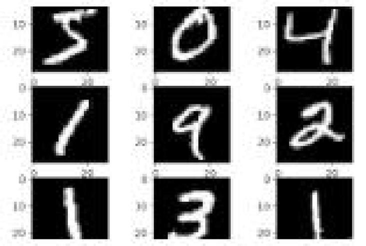


https://amitness.com/posts/images/simclr-random-transformation-function.gif

Source: amitness.com

https://albumentations.ai/img/basic_usage/choosing_augmentations/densification_vs_stress.webp

Source: albumentations.ai



https://www.machinelearningmastery.com/wp-content/uploads/2019/02/Plot-of-a-Subset-of-Images-from-the-MNIST-Dataset.png

Source: machinelearningmastery.com

https://www.nomidl.com/wp-content/uploads/2023/05/image-3.png

Source: www.nomidl.com